COPY-ON-WRITE MEMORY DEMONSTRATOR

- P NEEHARIKA-2520090233
- G SHIVANI-2520090046
- Department: Computer Science and Information Technology

TECHNOLOGY: C PROGRAMMING | LINUX | GCC

PROBLEM STATEMENT

WHEN A PROCESS CREATES A CHILD USING FORK(), COPYING ALL ITS MEMORY IMMEDIATELY CAN BE EXPENSIVE.

CHALLENGES:

- UNNECESSARY MEMORY COPYING CAN INCREASE OVERHEAD.
- UNDERSTANDING PROCESS MEMORY ISOLATION IS DIFFICULT FOR BEGINNERS.
- THE COPY-ON-WRITE MECHANISM IS NOT DIRECTLY VISIBLE TO USERS.

GOAL: DEVELOP A DEMONSTRATION SYSTEM SHOWING HOW MEMORY IS SHARED AFTER FORK() AND COPIED ONLY WHEN A PROCESS MODIFIES IT.

PROPOSED SOLUTION

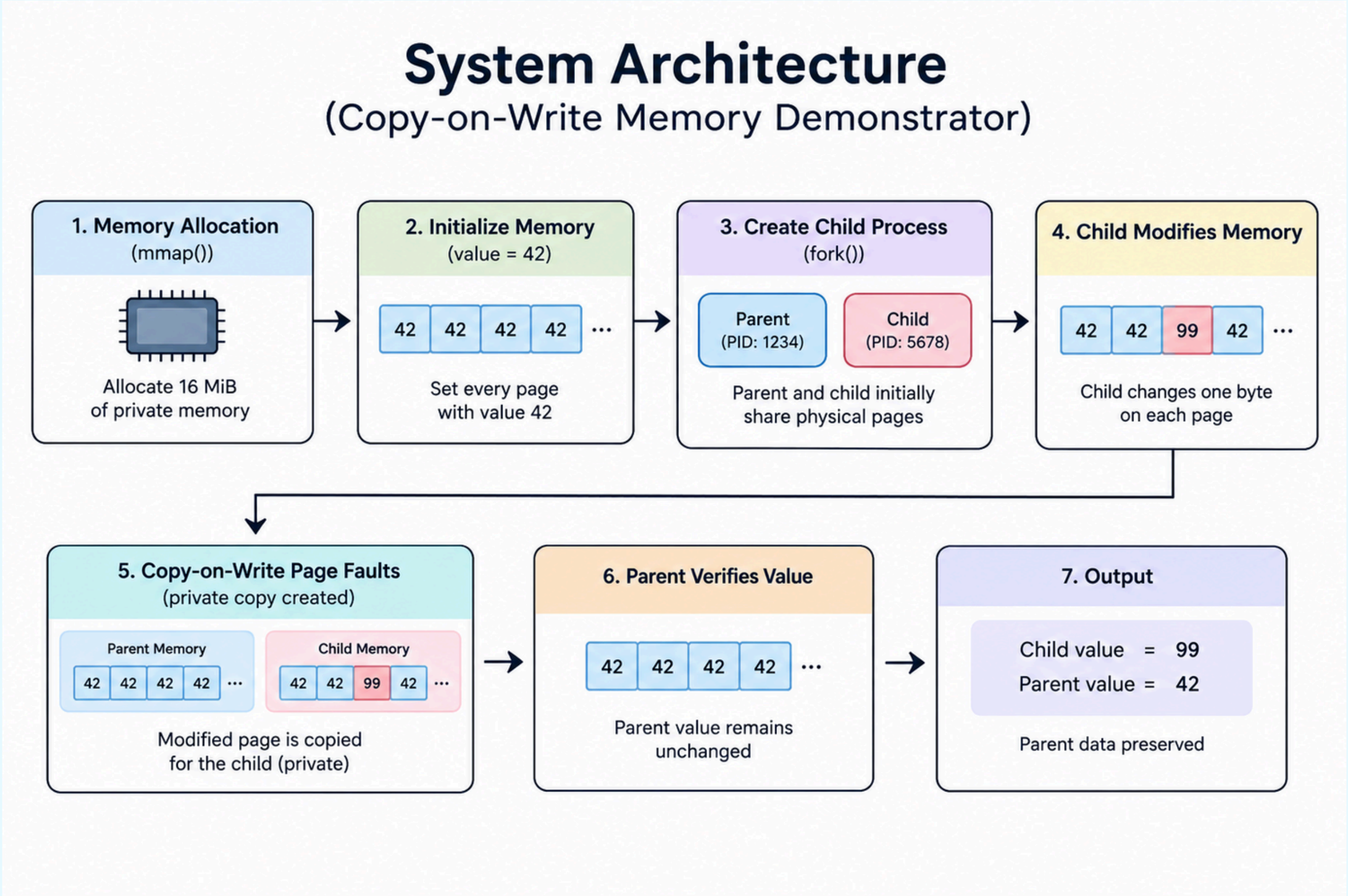
A C program that demonstrates Copy-on-Write behavior using Linux system calls.

Working:

- Allocate and initialize 16 MiB of memory.
- Create a child process using fork().
- Modify memory locations in the child process.
- Verify that the parent's original data remains unchanged.

Novelty: A simple, repeatable, practical demonstration of an internal operating-system memory-management concept using observable output and page-fault statistics.

SYSTEM ARCHITECTURE



IMPLEMENTATION

Programming Language: C
Platform: Linux / Ubuntu (WSL)
Tools: GCC Compiler, VS Code
System Calls and Functions:

- mmap() — allocates private anonymous memory.
- fork() — creates the child process.
- waitpid() — waits for the child to finish.
- getrusage() — retrieves resource usage, including minor page faults.

Memory Configuration:

- Allocated memory: 16 MiB
- Page size: 4096 bytes
- Initialized pages: 4096

TESTING AND RESULTS

TESTEXPECTED / OBSERVED RESULT

MEMORY INITIALIZATION	VALUE = 42
CHILD MODIFIES MEMORY	VALUE = 99
PARENT CHECKS MEMORY	VALUE = 42
PARENT DATA PRESERVED	YES
CHILD TERMINATION	NORMAL

```
root@HP:~/cow-memory-demonstrator# ./cow_demo | tee results/run1.txt
=== Copy-on-Write Memory Demonstrator ===
Page size: 4096 bytes
Memory allocated: 16 MiB
Pages initialized: 4096
Original value: 42

[PARENT PROCESS]

[CHILD PROCESS]Parent PID: 1020
Child PID: 1022

Child PID: 1022
Parent PID: 1020
Value before modification: 42
Value after modification: 99
Child minor faults during writes: 4096
Child changed 4096 page locations.
Child exits.

[FINAL VERIFICATION]
Parent value: 42
Expected parent value: 42
Parent data preserved: YES
Parent minor faults since fork: 7
Child exited normally: YES
root@HP:~/cow-memory-demonstrator#
```

RESULT: THE CHILD CAN MODIFY ITS MEMORY WITHOUT CHANGING THE PARENT'S ORIGINAL VALUE.
THE OBSERVED MINOR PAGE FAULTS PROVIDE SUPPORTING EVIDENCE OF PAGE-FAULT ACTIVITY DURING WRITES

IMPACT AND FUTURE SCOPE

Impact

- Improves understanding of operating-system memory management.
- Demonstrates process creation and memory isolation.
- Provides a reproducible learning and testing tool.

Future Enhancements

- Compare different memory allocation sizes.
- Run repeated experiments and summarize results.
- Visualize page-fault statistics.
- Compare memory behavior before and after writes.

CONCLUSION AND LIVE DEMO

Conclusion

- Successfully implemented a Linux-based COW demonstrator.
- Demonstrated parent-child process memory isolation.
- Verified that the child changes its value to 99 while the parent retains 42.
- Created a foundation for further memory-management experiments.

Live Demo: Compile and execute cow_demo.c in Ubuntu.

GitHub Repository:

https://github.com/mynamesteenzz-droid/cow-memory-demonstrator

Thank You!

THANK YOU